

Universidad Nacional Autónoma de México

Facultad de Ciencias

Tarea 04

No. de cuenta	Nombre
320148204	González Lucero Alan Uriel
321225087	Rivera Jiménez Isaac
321306102	Tapia Sánchez Jesús



Complejidad Computacional

Semestre 2027-1

Índice

1 Ejercicio 1 **2**

2 Ejercicio 2 **4**

3 Ejercicio 3 **13**

4 Ejercicio 4 **19**

Ejercicio 1

Enunciado

Sean L_1, L_2 definidos sobre Σ^*. Definimos la operación $\oplus$ de la siguiente manera:

$$L_1 \oplus L_2 = \{x \mid x \text{ está en exactamente uno de los lenguajes, en } L_1 \text{ o (exclusivo) en } L_2\}$$

Supón que ambos lenguajes, L_1, L_2, pertenecen a las clases de complejidad NP y coNP. Prueba que $L_1 \oplus L_2 \in \text{NP} \cap \text{coNP}$.

Solución

Por definición de $L_1 \oplus L_2$, sabemos que las cadenas x que contiene dicho conjunto cumplen con que x sólo puede pertenecer en L_1 o L_2, no en ambos.
Sea x una cadena y $x \in L_1 \oplus L_2$ entonces tenemos los siguientes casos:
Caso 1 | $x \in L_1$
Si $x \in L_1$ por def. de $L_1 \oplus L_2$ tenemos que $x \notin L_2$.
De lo cual, por def. del complemento de un lenguaje:

$$\bar{L_2} = \Sigma^* - L_2 = \{x \mid x \in \Sigma^* \text{ y } x \notin L_2\} \Rightarrow \text{en particular } x \in \bar{L_2} \Rightarrow x \in L_1 \cap \bar{L_2}$$

Caso 2 | $x \in L_2$
Si $x \in L_2 \Rightarrow x \notin L_1 \Rightarrow x \in \bar{L_1} \Rightarrow x \in L_2 \cap \bar{L_1}$

C.O.C. | $x \in (L_1 \cap \bar{L_2}) \cup (L_2 \cap \bar{L_1})$
Por lo que, podemos concluir que $L_1 \oplus L_2 = (L_1 \cap \bar{L_2}) \cup (L_2 \cap \bar{L_1})$.

Por otra parte, el problema nos pide suponer que ambos lenguajes L_1, L_2 pertenecen a NP y coNP. Es decir:

$$L_1, L_2 \in \text{NP} \cap \text{coNP}$$

Entonces, por def. de intersección:

$$L_1, L_2 \in \text{NP} \text{ y } L_1, L_2 \in \text{coNP}$$

Como $L_1, L_2 \in \text{NP} \Rightarrow$ por def. de coNP $\bar{L_1}, \bar{L_2} \in \text{coNP}$.
Como $L_1, L_2 \in \text{coNP} \Rightarrow$ por def. de coNP $\bar{L_1}, \bar{L_2} \in \text{NP}$.
$\therefore \bar{L_1}, \bar{L_2} \in \text{NP} \cap \text{coNP}$.

Ahora bien, después de todo este análisis. Para demostrar que $L_1 \oplus L_2 \in \text{NP} \cap \text{coNP}$, tendremos que probar que $L_1 \oplus L_2 \in \text{NP}$ y $L_1 \oplus L_2 \in \text{coNP}$.

P.D. que $L_1 \veebar L_2 \in NP$.

$$L_1 \veebar L_2 = (L_1 \wedge \overline{L_2}) \vee (L_2 \wedge \overline{L_1})$$

Por las anteriores demostraciones, sabemos que $L_1 \in NP$ y $\overline{L_2} \in NP \Rightarrow$ aprovechando la cerradura bajo la intersección de NP $A = (L_1 \wedge \overline{L_2}) \in NP$.
De manera análoga, como $L_2 \in NP$ y $\overline{L_1} \in NP \Rightarrow B = (L_2 \wedge \overline{L_1}) \in NP$.

Como $A \in NP$ y $B \in NP \Rightarrow$ por la cerradura bajo la unión de NP $A \cup B \in NP$.
Por lo tanto:

$$L_1 \veebar L_2 = (L_1 \wedge \overline{L_2}) \vee (L_2 \wedge \overline{L_1}) \in NP \quad //$$

P.D. que $L_1 \veebar L_2 \in coNP$.
Para probar que un lenguaje pertenece a coNP, basta con demostrar que su complemento pertenece a NP.
Calculamos el complemento de $L_1 \veebar L_2$ utilizando las leyes de De Morgan:

$$\overline{L_1 \veebar L_2} = (L_1 \wedge \overline{L_2}) \vee (L_2 \wedge \overline{L_1})$$
$$= \overline{(L_1 \wedge \overline{L_2})} \wedge \overline{(L_2 \wedge \overline{L_1})}$$
$$= (\overline{L_1} \vee L_2) \wedge (\overline{L_2} \vee L_1)$$

Distribuyendo los términos:

$$\overline{L_1 \veebar L_2} = (\overline{L_1} \wedge \overline{L_2}) \vee \cancel{(\overline{L_1} \wedge L_1)^{\perp}} \vee \cancel{(L_2 \wedge \overline{L_2})^{\perp}} \vee (L_2 \wedge L_1)$$

Como $(\overline{L_1} \wedge L_1) = \perp$ y $(L_2 \wedge \overline{L_2}) = \perp$, simplificamos a:

$$\overline{L_1 \veebar L_2} = (L_1 \wedge L_2) \vee (\overline{L_1} \wedge \overline{L_2})$$

Por otro lado, sabemos que:

$$L_1, L_2 \in NP \Rightarrow (L_1 \wedge L_2) \in NP$$
$$\overline{L_1}, \overline{L_2} \in NP \Rightarrow (\overline{L_1} \wedge \overline{L_2}) \in NP$$

Entonces, utilizando la cerradura bajo la unión de NP:

$$(L_1 \wedge L_2) \vee (\overline{L_1} \wedge \overline{L_2}) \in NP \Rightarrow \overline{L_1 \veebar L_2} \in NP$$

$\therefore$ al ser el complemento un lenguaje en NP:

$$L_1 \veebar L_2 \in coNP \quad //$$

Conclusión.
Hemos demostrado que $L_1 \veebar L_2 \in NP$ y $L_1 \veebar L_2 \in coNP$, en particular:

$$L_1 \veebar L_2 \in NP \cap coNP$$

Ejercicio 2

Enunciado

¿Cuáles de los siguientes problemas pertenecen a P, NP ó coNP? ¿Cuáles a coNP? Justifica tu respuesta argumentando tu conjetura o dando la demostración de pertenencia a las clases correspondientes.
Nota: Considerar que al menos una demostración de pertenencia a NP se deberá realizar utilizando la noción de No Determinismo, y al menos una utilizando la definición alternativa (con la noción de verificación).

- **DOM** $= \{\langle G, K\rangle \mid G$ tiene un conjunto dominante de K nodos$\}$
  Un subconjunto de vértices C de G es un conjunto dominante de G, si todo otro vértice de G es adyacente a algún vértice de C.

Solución

Definición Formal del Lenguaje DOM

El lenguaje del problema del conjunto dominante en su versión de decisión se define como:

$\text{DOM} = \{\langle G, K\rangle \mid G$ es una gráfica no dirigida y G tiene un conjunto dominante de K nodos$\}$

Un subconjunto de vértices $C \subseteq V(G)$ es un **conjunto dominante** si $|C| \leq K$ y para todo vértice $v \in V(G) \setminus C$, existe al menos un vértice $u \in C$ tal que $\{u, v\} \in E(G)$.

Definición de NP por Verificación

Un lenguaje L pertenece a la clase **NP** si existen un polinomio $p(n)$ y una Máquina de Turing determinista M de tiempo polinomial tales que para toda cadena de entrada x:

$$x \in L \iff \exists u \in \{0,1\}^{p(|x|)} \text{ tal que } M(x, u) = 1$$

P.D. que el problema DOM pertenece a NP mediante un verificador determinista

1. Definición del Certificado (Testigo) u y Esquema de Codificación

- **Certificado (u):** Es el subconjunto candidato de vértices $C = \{v_1, v_2, \ldots, v_m\} \subseteq V(G)$.
- **Esquema de Codificación:** El certificado se codifica como una lista explícita de los índices binarios de los vértices seleccionados o como un vector binario de longitud $|V|$ sobre el alfabeto $\{0, 1\}$.
- **Cota Polinomial:** El tamaño del certificado cumple $|u| \leq O(|V| \log |V|)$, el cual está acotado por un polinomio $p(|x|)$ respecto a la longitud de la representación binaria de la entrada $x = |\langle G, K\rangle|$.

2. Algoritmo del Verificador Determinista $M(x, u)$

La máquina de Turing determinista M recibe la pareja de entrada $x = (G, K)$ y el certificado candidato $u = C$ y ejecuta la siguiente secuencia de pasos:

1. Calcula la cardinalidad del conjunto C. Si $|C| > K$ o si C contiene elementos fuera de $V(G)$, la máquina **rechaza** inmediatamente devolviendo `0`.
2. Inicializa un conjunto de nodos cubiertos $S = C$.
3. Para cada vértice $u \in C$, recorre sus listas de adyacencia en la gráfica G y agrega todos sus vecinos inmediatos al conjunto S.
4. Compara la cantidad de nodos en S con el total de vértices $|V(G)|$. Si $S = V(G)$ (es decir, todo nodo $v \notin C$ tiene al menos un vecino en C), la máquina **acepta** devolviendo `1`; de lo contrario, **rechaza** devolviendo `0`.

3. Análisis de Complejidad en Tiempo

– El análisis de cardinalidad y validez en el Paso 1 toma tiempo $O(|V|)$.

– El recorrido de los vértices de C y el marcado de sus vecinos en los Pasos 2 y 3 toma un tiempo proporcional al grado de los nodos examinados, acotado a lo más por el número de aristas de la gráfica $O(|V| + |E|)$.

– El Paso 4 consiste en comparar el número de nodos marcados con $|V|$, lo cual toma tiempo $O(|V|)$.

El tiempo total de ejecución del verificador M está acotado por $O(|V| + |E|)$ operaciones deterministas, lo cual representa una función polinomial en el tamaño del ejemplar de entrada $|\langle G, K\rangle|$.

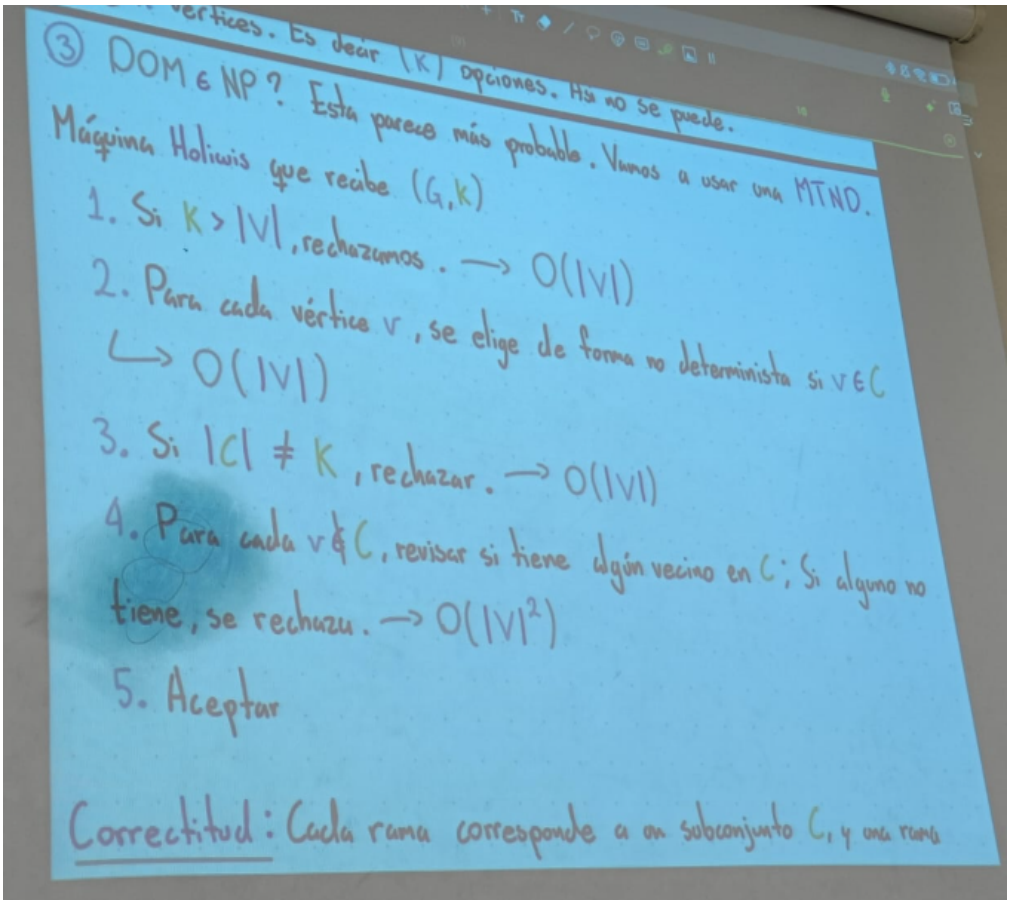
Conclusión

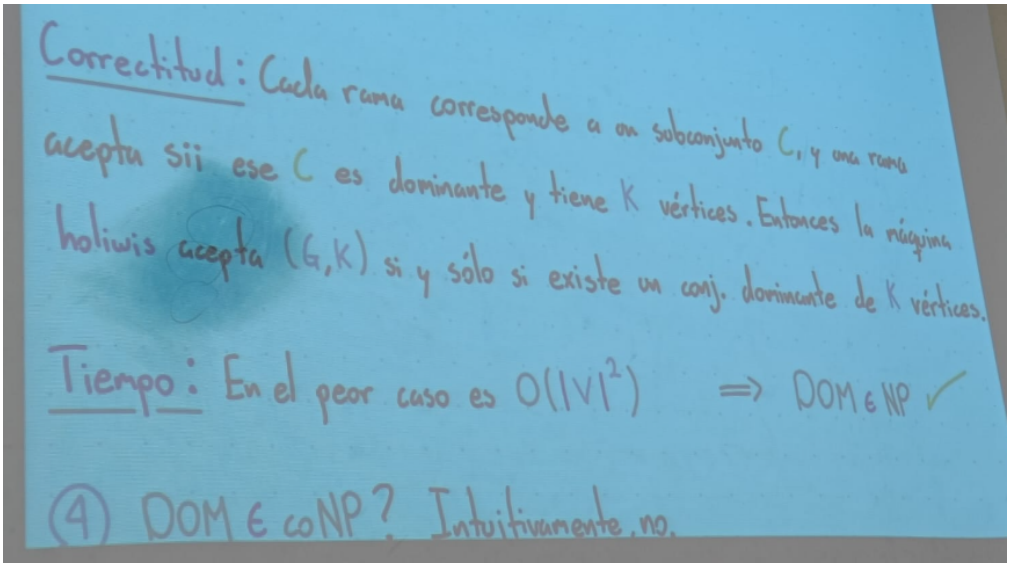
Si $(G, K) \in$ DOM, existe al menos un subconjunto de vértices C con $|C| \leq K$ que domina a G. Dicho conjunto sirve como certificado u de tamaño polinomial que hace que el verificador responda $M((G, K), C) = 1$.

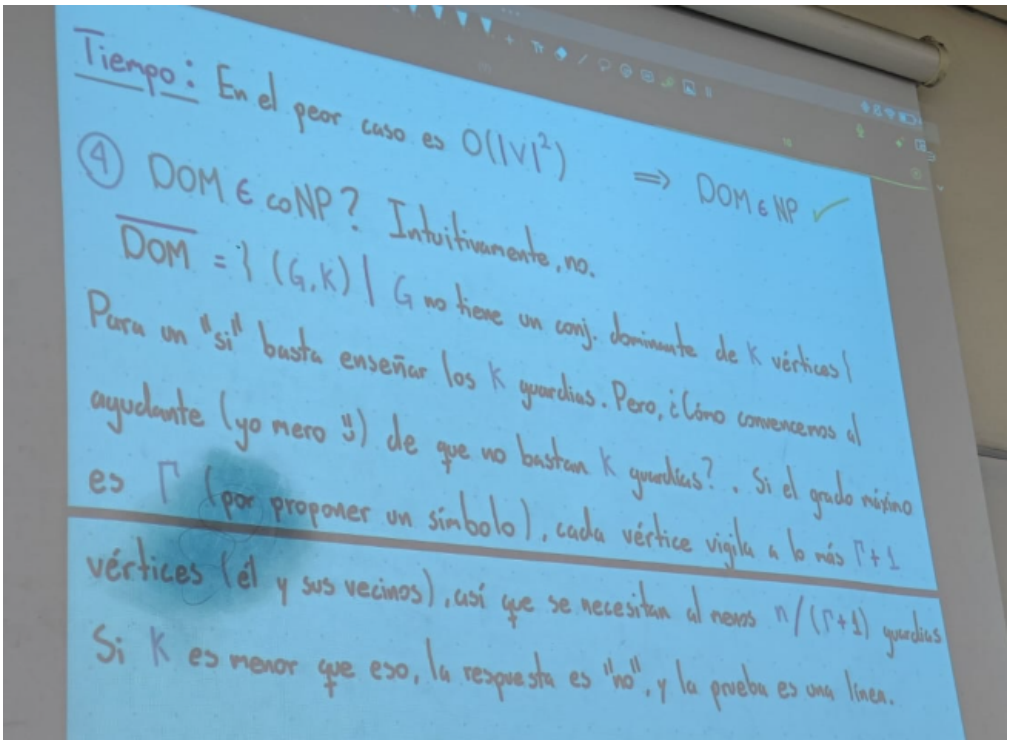
Si $(G, K) \notin$ DOM, no existe ningún conjunto de tamaño $\leq K$ capaz de dominar a G. Por ende, para cualquier certificado candidato u, el verificador responderá $M((G, K), u) = 0$.

Dado que se definió un certificado de tamaño polinomial y un verificador determinista de tiempo polinomial, queda demostrado que DOM $\in$ **NP**.

P.D. que el problema DOM pertenece a NP mediante una MTND


vértices. Es decir (K) opciones. Hay no se puede.
③ DOM ∈ NP? Esta parece más probable. Vamos a usar una MTND.
Máquina Holiwis que recibe (G,k)
1. Si K > |V|, rechazamos. → O(|V|)
2. Para cada vértice v, se elige de forma no determinista si v ∈ C
↳ O(|V|)
3. Si |C| ≠ K, rechazar. → O(|V|)
4. Para cada v ∉ C, revisar si tiene algún vecino en C; Si alguno no tiene, se rechaza. → O(|V|²)
5. Aceptar
Correctitud: Cada rama corresponde a un subconjunto C, y una rama


Correctitud: Cada rama corresponde a un subconjunto C, y una rama acepta sii ese C es dominante y tiene K vértices. Entonces la máquina holiwis acepta (G,K) si y sólo si existe un conj. dominante de K vértices.
Tiempo: En el peor caso es O(|V|²) ⇒ DOM ∈ NP ✓
④ DOM ∈ coNP? Intuitivamente, no.



- **L** = $\{\langle G\rangle \mid G$ es una gráfica no dirigida que NO contiene ciclos Eulerianos$\}$

Solución

Definición Formal del Lenguaje L

El lenguaje del problema se define como:

$$L = \{\langle G\rangle \mid G \text{ es una gráfica no dirigida que NO contiene ciclos Eulerianos}\}$$

Un **ciclo Euleriano** en una gráfica no dirigida es un ciclo que recorre cada arista de G exactamente una vez.

Teorema de Euler

Por el **Teorema de Euler**:

> Una gráfica conexa G contiene un ciclo Euleriano si y solo si el grado de todos sus vértices es **par**.

Por lo tanto, su condición de negación lógica establece que una gráfica G **NO contiene un ciclo Euleriano** si y solo si cumple al menos una de las siguientes dos condiciones:

1. Existe al menos un vértice $v \in V(G)$ cuyo **grado es impar** $(\text{grado}(v) \pmod 2 \neq 0)$.

2. La gráfica G es **desconexa** (sus vértices no aislados están divididos en dos o más componentes conexas distintas).

Clasificación del Problema

Gracias al teorema de Euler, el lenguaje L **pertenece a la clase P**, ya que es posible decidir determinísticamente si una gráfica no contiene ciclos Eulerianos mediante un algoritmo de tiempo polinomial.

P.D. que L pertenece a P mediante una máquina determinista decidible

1. Algoritmo Determinista de Decisión $M(\langle G \rangle)$

Dada la codificación de una gráfica no dirigida $G = (V, E)$:

- Recorre la lista de adyacencia o matriz de adyacencia de G y calcula el grado de cada vértice $v \in V$.
- Si encuentra algún vértice v con **grado impar**, la máquina **acepta** inmediatamente (devuelve `1`), confirmando que $G \in L$.
- Si todos los vértices tienen grado par, selecciona un vértice no aislado $s \in V$ y ejecuta BFS o DFS para marcar todos los vértices alcanzables desde s.
- Si el número de vértices marcados es menor que el número de vértices no aislados en G (lo que indica que la gráfica es desconexa), la máquina **acepta** (devuelve `1`).
- Si todos los vértices tienen grado par y pertenecen a una sola componente conexa, la gráfica G sí posee un ciclo Euleriano, por lo que la máquina **rechaza** (devuelve `0`).

2. Análisis de Complejidad en Tiempo

- El cálculo de los grados de todos los vértices toma tiempo $O(|V| + |E|)$.
- La comprobación de grados impares requiere recorrer los $|V|$ valores de grado, lo cual toma tiempo $O(|V|)$.
- El algoritmo BFS/DFS para verificar la conectividad de la gráfica se ejecuta en tiempo $O(|V| + |E|)$.

El tiempo total de ejecución es $O(|V|+|E|)$, el cual está acotado por un polinomio respecto al tamaño de la entrada $|\langle G \rangle|$. Por ende:

$$L \in \mathbf{P}$$

Conclusión

Dado que la clase determinista **P** está contenida tanto en **NP** como en **coNP** ($\mathbf{P} \subseteq \mathbf{NP} \cap \mathbf{coNP}$):

Como $L \in \mathbf{P}$, se puede usar el algoritmo determinista M como un verificador trivial que ignora cualquier certificado, lo que demuestra que $L \in \mathbf{NP}$.

Por otra parte, al ser $\mathbf{P}$ cerrada bajo complemento ($\mathbf{P} = \text{co}\mathbf{P}$), el complemento de L también está en $\mathbf{P}$, lo que implica directamente que $L \in \mathbf{coNP}$.

- **HAM_MOD** $= \{\langle G_1, G_2\rangle \mid G_1$ tiene un ciclo Hamiltoniano y G_2 no tiene un ciclo Hamiltoniano$\}$

Solución

Clasificación del Problema

A diferencia de los otros problemas que se han trabajado en clase directamente o tan siquiera se ha mencionado su respectiva clase de complejidad. Para esta ocasión nos enfrentamos con una modificación en la que si bien parte de un problema que ya conocemos, dicho cambio se realizó con maña para recordar que existen otras clases de complejidad a parte de **P**, **NP** y **coNP**.

Siendo que, decidimos clasificar a HAM_MOD dentro de la clase de tiempo exponencial determinista (**EXP**) de acuerdo a las siguientes razones (la idea nos la dio el profesor al final de la clase del día 27/10/26, simplemente la desarrollamos de una manera más formal y propia):

Por qué NO pertenece a P

Para que un problema pertenezca a la clase **P**, debe existir un algoritmo determinista capaz de decidir cualquier ejemplar en tiempo polinomial. En el caso de HAM_MOD:

- No se conoce ningún algoritmo determinista en tiempo polinomial, propiedad estructural ni teorema eficiente (como el Teorema de Euler para ciclos eulerianos) que permita decidir si una gráfica general posee o no un ciclo Hamiltoniano.
- Dado que el problema del Ciclo Hamiltoniano es un problema emblemático **NP**, determinar si G_1 tiene ciclo y G_2 no lo tiene requeriría resolver dos problemas intratables en un solo paso determinista. Por ende, **no hay evidencia de que HAM_MOD** $\in \mathbf{P}$.

La componente del problema que pertenece en NP (G_1)

La primera condición de la pareja exige que la gráfica G_1 posea un ciclo Hamiltoniano. Esta parte del problema pertenece a la clase **NP**, ya que si la respuesta para G_1 es afirmativa, existe un **certificado corto** (la permutación ordenada de los vértices de G_1) que un verificador determinista puede comprobar en tiempo polinomial $O(|V_1|)$.

La componente del problema que pertenece en coNP (G_2)

La segunda condición exige que la gráfica G_2 **NO** posea ningún ciclo Hamiltoniano. Esta condición pertenece a la clase **coNP** (el complemento de un problema **NP**). Para confirmar

que G_2 no tiene ciclos de forma determinista o para descalificar una respuesta falsa, se evalúa con el cuantificador universal ($\forall$), donde se debe garantizar que *ninguna* de las posibles permutaciones de vértices forme un ciclo válido.

Por qué no cabe únicamente en NP ni en coNP

El lenguaje HAM_MOD combina simultáneamente una condición existencial ($\exists$ para G_1) y una condición universal ($\forall$ para G_2) sobre dos instancias independientes. Si quisiéramos clasificar a HAM_MOD dentro de **NP**, necesitaríamos un certificado polinomial para probar que G_2 *no* tiene ciclo Hamiltoniano, lo cual requeriría que **coNP** $\subseteq$ **NP**. De igual forma, si quisiéramos clasificarlo únicamente en **coNP**, requeriríamos que **NP** $\subseteq$ **coNP**. Bajo la conjetura ampliamente aceptada de que **NP** $\neq$ **coNP**, un problema que combina ambas condiciones no puede ser contenido de forma única en ninguna de las dos clases, quedando fuera de la unión de **NP** $\cup$ **coNP**.

Conclusión del análisis

Dado que tanto **NP** como **coNP** están contenidas en la clase de tiempo exponencial determinista (**NP** $\subseteq$ **EXP** y **coNP** $\subseteq$ **EXP**), cualquier problema definido por este tipo de condiciones se puede **decidir mediante un algoritmo determinista de fuerza bruta**.

P.D. que HAM_MOD pertenece a EXP mediante una máquina determinista decidible

1. **Construcción del Decisor Determinista** $M((G_1, G_2))$**:**
   - **Fase 1 (Verificar** G_1**):** Explora de forma determinista las $|V_1|!$ posibles permutaciones de vértices de G_1. Si encuentra al menos una permutación que forme un ciclo Hamiltoniano válido, continúa con la fase 2; si explora todas sin éxito, **Rechaza**.
   - **Fase 2 (Verificar** G_2**):** Explora de forma determinista las $|V_2|!$ posibles permutaciones de vértices de G_2. Si comprueba que **ninguna** permutación forma un ciclo Hamiltoniano, **Acepta**; si encuentra alguna que sí forma ciclo, **Rechaza**.
2. **Análisis de Complejidad de Tiempo:**
   La exploración de G_1 toma tiempo $O(|V_1|! \cdot |V_1|) = O(2^{c_1 |V_1| \log |V_1|})$.

   La exploración de G_2 toma tiempo $O(|V_2|! \cdot |V_2|) = O(2^{c_2 |V_2| \log |V_2|})$.

   El tiempo total del algoritmo es la suma de ambos tiempos exponenciales:

   $$T(n) = O(2^{n^k})$$

   donde $n = |\langle G_1, G_2 \rangle|$ es el tamaño de la entrada.

Conclusión final

Dado que no existe un algoritmo de tiempo polinomial determinista conocido, y al combinar dos condiciones asimétricas (**NP** y **coNP**) sobre instancias distintas que pueden resolverse exhaustivamente en tiempo exponencial, se conjetura que la clase de complejidad adecuada para HAM_MOD es **EXP**.

- **CONECTIVIDAD** $= \{\langle G \rangle \mid G \text{ es una gráfica conexa}\}$

Solución

Definición Formal del Lenguaje CONECTIVIDAD

El lenguaje del problema se define en su versión de decisión como:

$$\text{CONECTIVIDAD} = \{\langle G \rangle \mid G \text{ es una gráfica no dirigida y conexa}\}$$

Una gráfica no dirigida $G = (V, E)$ es **conexa** si para todo par de vértices $u, v \in V$, existe al menos un camino (ruta) en G que conecta a u con v.

Clasificación de Complejidad

El problema **CONECTIVIDAD pertenece a la clase P**.

Dado que se cuenta con algoritmos deterministas de exploración de gráficas que pueden verificar la conexidad en tiempo lineal respecto al número de vértices y aristas, el problema es decidible en tiempo polinomial determinista.

P.D. que CONECTIVIDAD pertenece a P mediante una máquina determinista decidible

1. Algoritmo Determinista de Decisión $M(\langle G \rangle)$

Dada la codificación estándar de una gráfica no dirigida $G = (V, E)$ (por ejemplo, mediante listas de adyacencia):

- Si $0 \leq |V| \leq 1$, la gráfica es trivialmente conexa, por lo que la máquina **Acepta** (devuelve `1`).
- Selecciona un vértice inicial arbitrario $s \in V$.
- Inicializa un arreglo de marcas o un conjunto de nodos visitados $S = \{s\}$.
- Ejecuta BFS o DFS comenzando en s:
  - Mientras existan nodos no visitados, toma un nodo $u \in S$ no procesado, examina todos sus vecinos v en G y los añade al conjunto S.

- Compara la cardinalidad del conjunto de nodos visitados $|S|$ con el total de vértices $|V|$.
- Si $|S| = |V|$ (es decir, todos los nodos de G fueron alcanzados desde s), la máquina **Acepta** (devuelve `1`).
- Si $|S| < |V|$ (existen nodos inalcanzables desde s), la máquina **Rechaza** (devuelve `0`).

2. Análisis de Complejidad de Tiempo

La inicialización toma tiempo $O(|V|)$.

El algoritmo de exploración (BFS/DFS) visita cada vértice a lo más una vez y examina cada arista dos veces (una por cada uno de sus extremos en una gráfica no dirigida).

Por lo tanto, el tiempo total de ejecución está acotado por:

$$T(n) = O(|V| + |E|)$$

Como $|V|+|E|$ es proporcional a la longitud de la cadena de entrada $n = |\langle G \rangle|$, la función de complejidad de tiempo es polinomial $O(n)$. Por ende:

$$\text{CONECTIVIDAD} \in \mathbf{P}$$

Conclusión

Al ser un lenguaje perteneciente a la clase determinista **P**:

Como $\mathbf{P} \subseteq \mathbf{NP}$, el algoritmo determinista M actúa como un verificador que ignora cualquier certificado, manteniendo la complejidad de tiempo $O(|V| + |E|)$.

Por otro lado, como la clase **P** es cerrada bajo complemento ($\mathbf{P} = \text{co}\mathbf{P}$), el complemento $\overline{\text{CONECTIVIDAD}}$ (las gráficas desconexas) también pertenece a **P**, concluyendo que **CONECTIVIDAD** $\in$ **coNP**.

- **(*) CONJ_INDEP** $= \{\langle G, k \rangle \mid$ la gráfica G tiene un conjunto independiente de tamaño al menos $k\}$
  ** Un conjunto independiente de los vértices de una gráfica G es un conjunto tal que ninguno de sus vértices es adyacente a otro.*

Solución

- **(*) K-COLORACIÓN** $= \{\langle G \rangle \mid G$ tiene una coloración de sus vértices, con K colores$\}$
  Investiga qué es una coloración en gráficas.

Solución

- **(*) EC-CUADRATICAS** $= \{\langle a, b, c \rangle \mid a, b, c \in \mathbb{Z}^+, \exists x, y \in \mathbb{Z}^+$ tal que $ax^2 + by = c\}$

Solución

** Opcional para equipos de 3 o menos integrantes. Equipos de 4 integrantes es obligatorio.*

Ejercicio 3

Enunciado

Considera el siguiente acertijo.
Bruno, Juan y Samuel trabajan en una ciudad como bombero, policía y maestro, aunque no necesariamente en ese orden.
En la ciudad se rumora lo siguiente:

- Bruno y el maestro son vecinos
- Juan y el maestro son vecinos
- Bruno y Samuel (ambos) son vecinos del bombero
- El policía y el bombero (ambos) son vecinos de Juan
- Todos son vecinos entre sí

Se sabe que solo dos de estas afirmaciones son verdaderas. ¿Qué oficio desempeña cada uno?

1. Expresa el acertijo anterior como un ejemplar concreto de algún problema en NP; Enuncia el problema en su versión canónica.
   Nota: Si el problema elegido no es uno de los vistos en clase, demuestra que el problema está en NP.

Solución

Primero sabemos por las proposiciones que la ultima sucede, pues todos son vecinos, sin embargo, sucede que el policia y el bombero son vecinos de Juan, de este modo

Nota: Si el problema elegido no es uno de los vistos en clase, demuestra que el problema está en NP.

Se pueden tener varias proposiciones logicas para este problema; visto de esta forma, se tiene la siguiente estructura:

(a) $p \to q$

(b) $m \to q$

(c) $B \land S \to b$

(d) $p \land b \to J$

(e) $b \land q \land p$

El problema puede ser visto como un problema de proposiciones logicas, ya que se busca que dos de las afirmaciones sean ciertas o satisfacibles del conjunto de proposiciones.

De la misma manera, dado que son 3 personas, se le puede asignar un posible oficio a cada una de ellas para determinar cual es el verdadero.

$$Bm, Bp, Bb, Sm, Sp, Sb, Jm, Jp, Jb$$

Por lo que el ejemplar para las proposiciones anteriores seria el siguiente:

Ejemplar concreto de SAT (Opcion 1)
Datos: Proposiciones logicas en conjuncion booleana, $p \to q$, $m \to q$, $B \wedge S \to b$, $p \wedge b \to J$, $b \wedge q \wedge p$
Respuesta: $(\neg p \vee q) \wedge (\neg m \vee q) \wedge (\neg B \vee \neg S \vee b) \wedge (\neg p \vee \neg b \vee J) \wedge (b \wedge q \wedge p) = 1$

Para la solución única del acertijo, definimos los valores de verdad de las variables atómicas según las profesiones asignadas:

- $p = 0$: Bruno no es policía; el policía es Juan.
- $m = 0$: Juan no es maestro; el maestro es Bruno.
- $b = 0$: Juan no es bombero; el bombero es Samuel.
- $q = 1$: Todos son vecinos entre sí.
- $B = 1$, $S = 1$, $J = 1$: Representan la presencia de Bruno, Samuel y Juan como entidades distintas.

Por lo que al evaluarlas tenemos lo siguiente:

(a) $p \to q \equiv 0 \to 1 \equiv 1$ (Verdadero)

(b) $m \to q \equiv 0 \to 1 \equiv 1$ (Verdadero)

(c) $B \wedge S \to b \equiv (1 \wedge 1) \to 0 \equiv 1 \to 0 \equiv 0$ (Falso)

(d) $p \wedge b \to J \equiv (0 \wedge 0) \to 1 \equiv 0 \to 1 \equiv 1$ (Verdadero)

(e) $b \wedge q \wedge p \equiv 0 \wedge 1 \wedge 0 \equiv 0$ (Falso)

De esta forma Bruno = Maestro, Juan = Policía, Samuel = Bombero, y la proposicion e) y b) son las unicas verdaderas.

Ejemplar concreto de SAT (Opcion 2)
Datos: Proposiciones logicas en conjuncion booleana, $\neg Bp$, $\neg Jm$, $\neg Jb$, $\neg Bp \wedge \neg Jm \wedge \neg Jb$, $b \wedge q \wedge p$

Respuesta: $(\neg p \vee q) \wedge (\neg m \vee q) \wedge (\neg B \vee \neg S \vee b) \wedge (\neg p \vee \neg b \vee J) \wedge (b \wedge q \wedge p) = 1$
Lo definimos como posibles opciones.

(a) $A_1 \equiv \neg B_m$ Bruno y el maestro son vecinos $\implies$ Bruno no es el maestro

(b) $A_2 \equiv \neg J_m$ (Juan y el maestro son vecinos $\implies$ Juan no es el maestro)

(c) $A_3 \equiv (\neg B_b \wedge \neg S_b) \leftrightarrow J_b$ Bruno y Samuel son vecinos del bombero $\implies$ El bombero es Juan

(d) $A_4 \equiv \neg J_p \wedge \neg J_b$ (El policía y el bombero son vecinos de Juan $\implies$ Juan no es policía ni bomber

(e) $A_5 \equiv \text{True}$ (Todos son vecinos entre sí $\implies$ Topología completa del vecindario)

Se construye la fórmula proposicional que exige que exactamente dos de las variables $\{A_1, A_2, A_3, A_4, A_5\}$ sean verdaderas y las otras tres sean falsas:

$$\Gamma_2 = \bigvee_{1 \le i < j \le 5} \left(A_i \wedge A_j \wedge \bigwedge_{k \notin \{i,j\}} \neg A_k \right)$$

Por lo tanto para este caso el ejemplar corresponderia a

$$\Gamma = \Gamma_1 \wedge \Gamma_2$$

, donde Γ_1 es toda la proposicion que es equivale a encontrar una interpretación $v : \{B_b, \ldots, S_m\} \to \{0, 1\}$ tal que Γ sea satisfacible ($\Gamma = 1$). Esto nos dara el mismo resultado que la proposicion anterior, por lo tanto funciona de igual manera.

Forma canonica: SAT
Datos: Un conjunto de variables proposicionales $V = \{x_1, x_2, \ldots, x_n\}$ y una fórmula lógica Γ expresada como una conjunción booleana de m cláusulas

$$\Gamma = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

, donde cada cláusula $C_i = (l_{i,1} \vee l_{i,2} \vee \cdots \vee l_{i,k_i})$ es una disyunción de literales con $l \in \{x_j, \neg x_j\}$. **Pregunta:** ¿Existe algún conjunto de valores $V \to \{0, 1\}$ para los cuales la formula logica en conjunción booleana de m clausulas sea satisfacible, es decir $\Gamma = 1$?

(Que yo recuerde si, pero no esta de mas) Decimos que $L \in NP \iff \exists M$ Determinista $|\exists x, |u| = n^k \; M(x, a) = 1$. Por lo tanto proponemos un certificado para M

Decimos que $L \in NP \iff \exists M$ Determinista $|\exists x, |u| = n^k \; M(x, a) = 1$.

Proponemos como certificado u una asignación de valores de verdad para las n variables de la fórmula proposicional x de longitud N, para saber si es satisfacible, el número de variables n está acotado por la longitud del problema ($n \leq N$), su tamaño es consecuentemente polinomial ($|u| = O(N)$).

Proponemos un algoritmo determinista de verificacion (x, u)

```
result = 1
// Evaluar cada cláusula C_i bajo la asignación dada por u
for (i = 0; i < m; i++):
    eval_clausula = 0
    for cada literal L en la clausula C[i]:
        val = u[L.variable]
        if (L es negado):
            val = !val
        eval_clausula = eval_clausula || val

    result = result && eval_clausula

if (result == 1):
    aceptar
else:
    rechazar
```

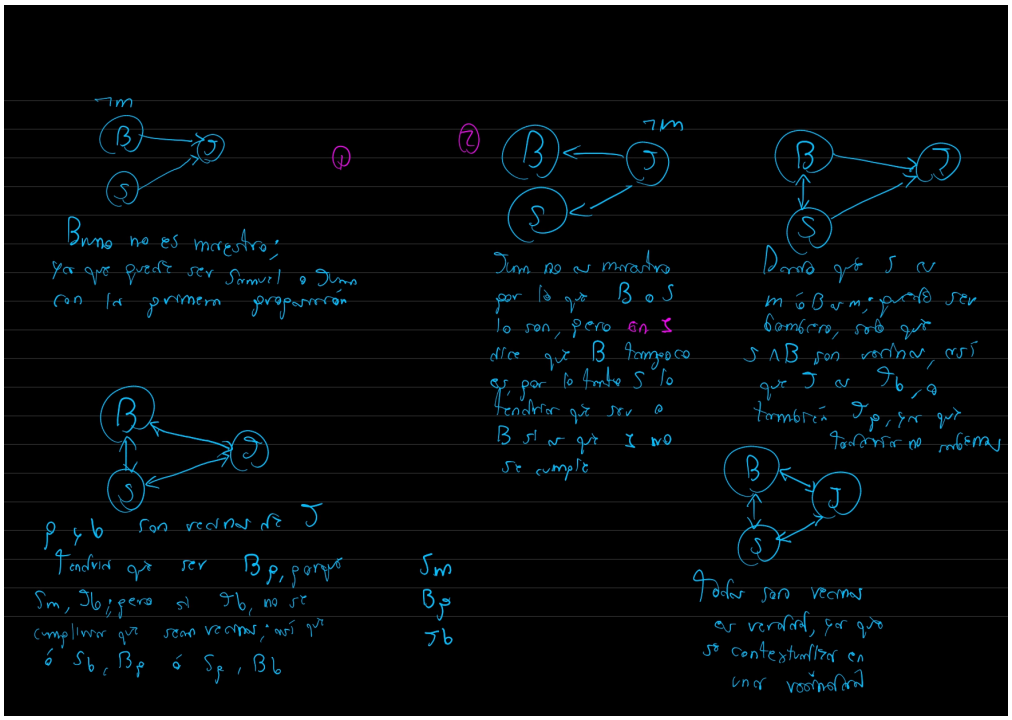
Asignarle un valor a cada variable atomica y comprobarlo toma $O(N^2)$ ya que recorrer dos veces las proposiciones y asignarles un valor toma $O(N^2)$, comparaciones dentro del for mas anidado toma $O(N^2)$, en el nivel más interno, se realiza una comparación de igualdad y operaciones booleanas (AND, OR, NOT), cada una de las cuales toma tiempo $O(1)$. Regresar valores $O(1)$, por lo tanto el algoritmo se ejecuta en un tiempo polinomial. Dado que en ambos escenarios la complejidad está acotada por un polinomio $O(N^k)$ (con $k \leq 2$), el algoritmo verificador $M(x, u)$ ejecuta en tiempo polinomial determinista, lo que demuestra que el problema pertenece a la clase NP.

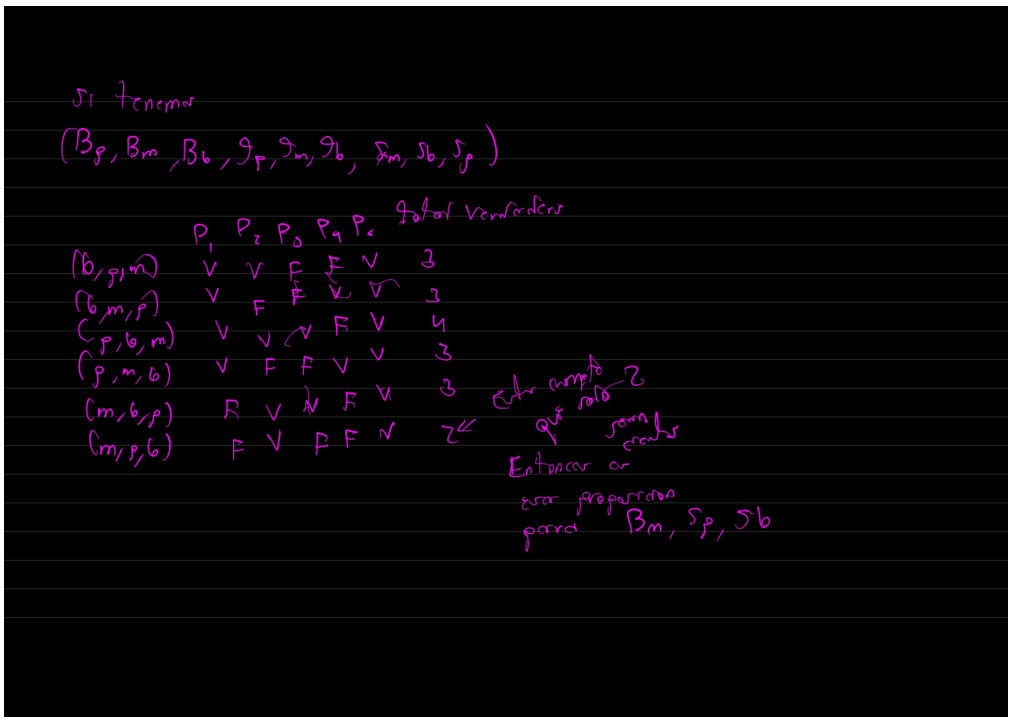
De esta manera **SAT** $\in NP$ para el certificado u.

2. Da una representación visual (diagrama, figura, imagen, etc..) del ejemplar concreto, de acuerdo al problema del inciso anterior.

Solución

Se puede mediante graficas representar el problema y cada proposicion.





3. Resuelve el acertijo exhibiendo un certificado.

Solución

Decimos que $L \in NP \iff \exists M$ Determinista $|\exists x, |u| = n^k \ M(x, a) = 1$.

Definimos el certificado u como el vector binario de asignación de verdad para las 9 variables proposicionales $u = (Bm, Bp, Bb, Jm, Jp, Jb, Sm, Sp, Sb) \in \{0, 1\}^9$:

$$u = (1, 0, 0, 0, 1, 0, 0, 0, 1)$$

Esta asignación codifica de forma única los siguientes oficios, Bruno: Maestro ($Bm = 1$)Juan: Policía ($Jp = 1$)Samuel: Bombero ($Sb = 1$)

Propuesto ya el verificador procesa el certificado u comprobando dos condiciones.

Satisfacción de los tipos de oficio de las personas ($\Gamma_1 = 1$)Cada persona tiene un único oficio:

$$B_m \vee Bp \vee Bb = 1 \vee 0 \vee 0 = 1$$

$$Jm \vee Jp \vee Jb = 0 \vee 1 \vee 0 = 1$$

$$Sm \vee Sp \vee Sb = 0 \vee 0 \vee 1 = 1$$

No hay solapamiento de oficios (todas las cláusulas de exclusión mutua se evalúan en 1). Por utlimo falta la evaluación del número de afirmaciones verdaderas ($\Gamma_2 = 1$) Sustituyendo la asignación dada por u en cada una de las 5 afirmaciones para el ejemplar 2:

(a) $A_1 \equiv \neg B_m \implies \neg(1) = 0$ (Falso)

(b) $A_2 \equiv \neg J_m \implies \neg(0) = 1$ (Verdadera)

(c) $A_3 \equiv (\neg B_b \wedge \neg S_b) \leftrightarrow J_b \implies (1 \wedge 0) \leftrightarrow 0 \implies 0 \leftrightarrow 0 \equiv 0$ (Falsa)

(d) $A_4 \equiv \neg J_p \wedge \neg J_b \implies \neg(1) \wedge \neg(0) = 0 \wedge 1 = 0$ (Falsa)

(e) $A_5 \equiv$ (esta ya era 1) $= 1$ (Verdadera)

Dado que el certificado u hace que la restricción de asignación de oficios se cumpla y genera exactamente 2 afirmaciones verdaderas (A_2 y A_5), la fórmula global evalúa en:

$$M(x, u) = \Gamma 1(u) \wedge \Gamma_2(u) = 1 \wedge 1 = 1$$

El algoritmo de verificación acepta el certificado u en tiempo $O(1)$, lo que demuestra que el ejemplar es satisfacible y resuelve el acertijo.

Ejercicio 4

Enunciado

Para cada uno de los lenguajes del ejercicio 2:
- Define el lenguaje correspondiente al complemento.
- ¿A qué clase de complejidad pertenece el complemento?
- Da un ejemplar concreto que pertenezca al complemento.

- **Problema DOM:**

Solución

1. Definición del Lenguaje Complemento ($\overline{\text{DOM}}$)

El complemento del lenguaje se define como $\overline{\text{DOM}} = \Sigma^* \setminus \text{DOM}$:

$\overline{\text{DOM}} = \{\langle G, K\rangle \mid G$ es una gráfica no dirigida que NO posee ningún conjunto dominante de K nodos$\}$

Un ejemplar (G, K) pertenece a $\overline{\text{DOM}}$ si y solo si la respuesta a la pregunta *"¿Tiene G un conjunto dominante de tamaño a lo más K?"* es **NO**.

Es decir, $(G, K) \in \overline{\text{DOM}}$ si **para todo** subconjunto de vértices $C \subseteq V(G)$ de tamaño $|C| \leq K$, existe al menos un vértice $v \in V(G) \setminus C$ que **no** es adyacente a ningún vértice de C.

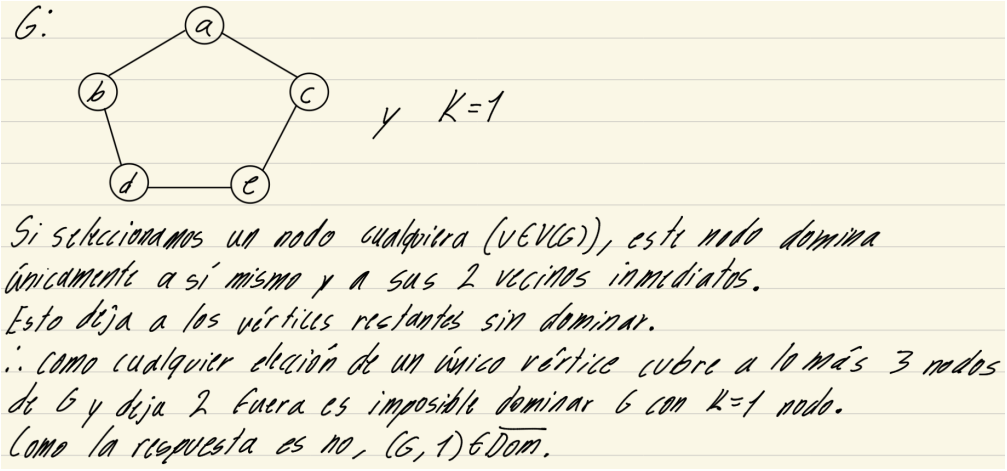
2. Clase de Complejidad a la que Pertenece $\overline{\text{DOM}}$

En el Ejercicio 2 se demostró que DOM $\in$ **NP**. Por definición de la clase complemento, si un lenguaje L pertenece a **NP**, su complemento $\overline{L}$ pertenece por definición a la clase **coNP**:

$$\text{coNP} = \{L \mid \overline{L} \in \textbf{NP}\}$$

Dado que DOM es **NP**, su complemento $\overline{\text{DOM}}$ es **coNP**.

3. Ejemplar Concreto que Pertenece a $\overline{\text{DOM}}$



- **Problema L (No ciclos Eulerianos):**

Solución

Definición del Lenguaje Complemento ($\overline{L}$)

El complemento del lenguaje se define como $\overline{L} = \Sigma^* \setminus L$:

$$\overline{L} = \{\langle G \rangle \mid G \text{ es una gráfica no dirigida que SÍ contiene un ciclo Euleriano}\}$$

Por el Teorema de Euler:
Para las cadenas bien formadas que representan una gráfica conexa G, $\langle G \rangle \in \overline{L}$ si y solo si **todos y cada uno de los vértices de G tienen grado par**.

Clase de Complejidad a la que Pertenece $\overline{L}$

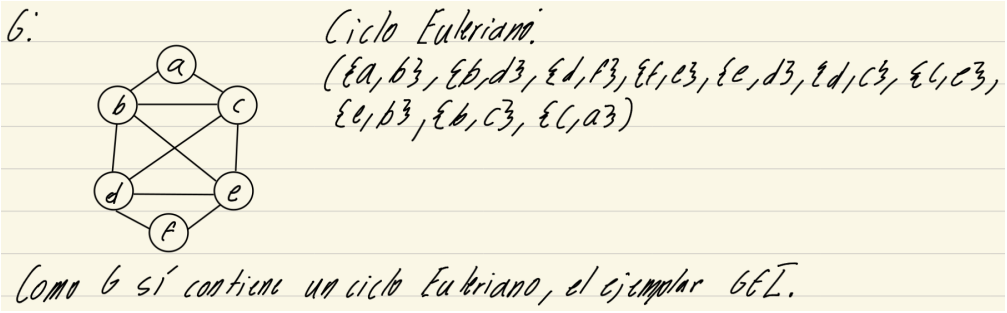
En el Ejercicio 2 demostramos que $L \in \mathbf{P}$ mediante un algoritmo determinista M que calcula el grado de cada vértice y comprueba la conexidad mediante BFS/DFS en tiempo $O(|V| + |E|)$.

Las clases de complejidad deterministas son **cerradas bajo la operación de complemento**:

$$\mathbf{P} = \text{co}\mathbf{P}$$

Por lo tanto, si $L \in \mathbf{P}$ (y por inclusión, también pertenece a **NP** y **coNP**), se concluye directamente que su complemento $\overline{L} \in \mathbf{P}$. El algoritmo determinista para $\overline{L}$ es idéntico al de L, invirtiendo únicamente los estados finales de aceptación y rechazo.

Ejemplar Concreto que Pertenece a $\overline{L}$



- **Problema HAM_MOD:**

Solución

Definición del Lenguaje Complemento ($\overline{\text{HAM_MOD}}$)

El complemento del lenguaje es $\overline{\text{HAM_MOD}} = \Sigma^* \setminus \text{HAM_MOD}$. Aplicando la regla de la negación lógica (Leyes de De Morgan) a la conjunción original:

$$\neg\Big((G_1 \in L_{\text{HAM}}) \wedge (G_2 \notin L_{\text{HAM}})\Big) \iff (G_1 \notin L_{\text{HAM}}) \vee (G_2 \in L_{\text{HAM}})$$

Por lo tanto, formalmente se define como:

$\overline{\text{HAM_MOD}} = \{\langle G_1, G_2\rangle \mid G_1$ NO tiene un ciclo Hamiltoniano $\vee\ G_2$ SÍ tiene un ciclo Hamiltoniano$\}$

Para que un ejemplar bien formado $\langle G_1, G_2\rangle$ pertenezca al complemento $\overline{\text{HAM_MOD}}$, la respuesta al problema original debe ser **NO**. Esto ocurre cuando se satisface **al menos una** de las siguientes dos situaciones:

1. La primera gráfica G_1 carece de ciclo Hamiltoniano.
2. La segunda gráfica G_2 sí posee un ciclo Hamiltoniano.

Clase de Complejidad a la que Pertenece $\overline{\text{HAM_MOD}}$

En el Ejercicio 2, dentro del marco del temario del curso, establecimos que HAM_MOD $\in$ **EXP** debido a que requiere decidir de manera exhaustiva dos propiedades intratables sobre dos gráficas distintas.

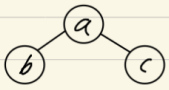
Dado que las clases de tiempo determinista como **EXP** son **cerradas bajo complemento** (**EXP** = coEXP), el complemento de cualquier lenguaje en **EXP** permanece dentro de la misma clase.

Un algoritmo determinista de fuerza bruta puede verificar en tiempo exponencial si G_1 no tiene ciclo o si G_2 sí lo tiene, concluyendo la pertenencia a **EXP**.

Ejemplar Concreto que Pertenece a $\overline{\text{HAM_MOD}}$

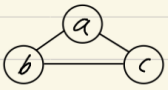
Como G sí contiene un ciclo Euleriano, el ejemplar $G \notin \overline{L}$.

G_1:



$\Rightarrow G_1$ no tiene ciclo Hamiltoneano.

G_2:



$\Rightarrow G_2$ sí tiene un ciclo Hamiltoneano.

Sea $x \subseteq \Sigma^* \ni x = (G_1, G_2)$, queremos saber si $x \in$ HAM_MOD.
* La primera gráfica G_1 falla la condición de tener un ciclo Hamiltoneano.
* La segunda gráfica G_2 falla la condición de no tener un ciclo Hamiltoneano.
Basta con que alguna de las condiciones falle (aunque mostramos ambas) para que la respuesta a la pregunta de decisión sea NO.
$\therefore (G_1, G_2) \in \overline{\text{HAM_MOD}}$.

- **Problema CONECTIVIDAD:**

Solución

Definición Formal del Lenguaje Complemento ($\overline{\text{CONECTIVIDAD}}$)

El complemento del lenguaje es $\overline{\text{CONECTIVIDAD}} = \Sigma^* \setminus \text{CONECTIVIDAD}$:

$$\overline{\text{CONECTIVIDAD}} = \{\langle G \rangle \mid G \text{ es una gráfica no dirigida que NO es conexa (desconexa)}\}$$

Una gráfica no dirigida bien formada $G = (V, E)$ pertenece a $\overline{\text{CONECTIVIDAD}}$ si es **desconexa**, lo cual significa que existen al menos dos vértices $u, v \in V$ entre los cuales no existe ningún camino en G (es decir, la gráfica se compone de dos o más componentes conexas disjuntas).

Clase de Complejidad a la que Pertenece ($\overline{\text{CONECTIVIDAD}}$)

En el Ejercicio 2 demostramos que CONECTIVIDAD $\in \mathbf{P}$ mediante un algoritmo determinista de Búsqueda en Anchura (BFS) o Profundidad (DFS) que explora la gráfica a partir de un nodo arbitrario s en tiempo lineal $O(|V| + |E|)$.

Las clases deterministas de complejidad de tiempo son **cerradas bajo la operación de complemento**:

$$\mathbf{P} = \text{co}\mathbf{P}$$

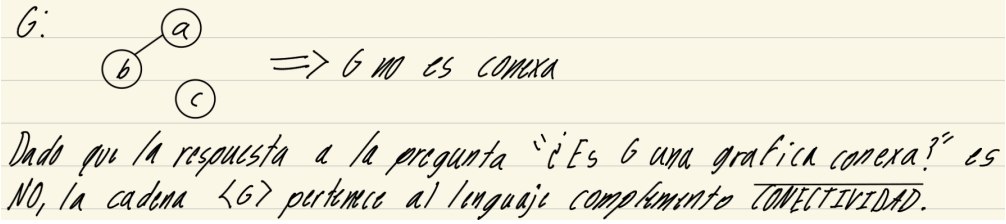
Por lo tanto, si CONECTIVIDAD $\in$ **P**, se concluye directamente que su complemento $\overline{\textbf{CONECTIVIDAD}} \in$ **P** (y por inclusión, también pertenece a **NP** y **coNP**).

El algoritmo determinista para $\overline{\text{CONECTIVIDAD}}$ ejecuta la misma exploración (BFS/DFS):

- Si la cantidad de nodos visitados desde s es estrictamente menor que $|V|$, la máquina **Acepta** (devuelve `1`), confirmando que la gráfica es desconexa.
- Si la cantidad de nodos visitados es igual a $|V|$, la máquina **Rechaza** (devuelve `0`).

Como este procedimiento determinista toma tiempo $O(|V| + |E|)$, el problema se decide en tiempo polinomial.

Ejemplar Concreto que Pertenece a $\overline{\text{CONECTIVIDAD}}$



- **Problema CONJ_INDEP (Opcional *):**

Solución

- **Problema K-COLORACIÓN (Opcional *):**

Solución

- **Problema EC-CUADRATICAS (Opcional *):**

Solución